ABI Go, обобщения и компоновщик

Физическая граница вызова, скрытая информация обобщенных функций и сборка исполняемого файла.

```
вызов → ABI → регистры / стек
обобщения → форма / словарь
символы → компоновщик → бинарник
```

Машинный код функции — еще не готовая программа

УПРОЩЕНИЕ

В 4.3 дошли до инструкций отдельной функции. На первый взгляд, работа закончена.

ТОЧНАЯ МОДЕЛЬ

Аргументы нужно физически передать; обобщенному коду нужны сведения о типе; ссылки между функциями еще предстоит связать.

ИНЖЕНЕРНЫЙ ВЫВОД

После компилятора остается межфункциональный и межпакетный слой.

Три вопроса, которые закрывает 4.4

ЛОГИКА ЛЕКЦИИ

```
функция A вызывает B
↓ где лежат аргументы и результат?
ABI
↓ как передать сведения о T?
форма + словарь
↓ как найти реальные адреса функций?
компоновщик
```

ИНЖЕНЕРНЫЙ ВЫВОД

Три механизма продолжают друг друга, а не существуют изолированно.

Почему нельзя прыгнуть от Go-кода сразу к исполняемому файлу

НИЗКИЙ УРОВЕНЬ

Машинные инструкции умеют загружать регистры, писать память и переходить по адресам.

СКРЫТЫЕ ДОГОВОРЕННОСТИ

Типовая система Go, обобщенные функции, динамические методы и внешние пакеты требуют дополнительных двоичных соглашений.

ИНЖЕНЕРНЫЙ ВЫВОД

Сборка исполняемой программы — больше, чем перевод операций в машинные инструкции.

Граница этой лекции

НЕ ЧИТАЕМ

Не изучаем подробный формат ELF/Mach-O, алгоритмы загрузчика ОС, PLT/GOT и все виды релокаций.

ИССЛЕДУЕМ

Открываем ABI вызовов, форма/словарь/обертки и граф символов компоновщика. Загрузчик ОС — следующая тема.

ИНЖЕНЕРНЫЙ ВЫВОД

Глубина определяется новым механизмом, а не каталогом системных терминов.

Кто кладет аргументы и кто их забирает

У функции есть сигнатура Go; CPU видит только регистры, память и инструкции.

```
логический вызов ≠ физический вызов
вызывающая функция ↔ вызываемая функция
ABIInternal / ABI0
```

Что скрывается за `add(a, b)`

GO / КОМАНДА

```
func add(a, b int) int {
    return a + b
}
```

ЧТО ПРОИСХОДИТ

Сигнатура говорит о двух значениях и результате, но не сообщает, в какой регистр или ячейку стека их положить.

ИНЖЕНЕРНЫЙ ВЫВОД

Между вызывающей и вызываемой функциями нужен двоичный контракт.

API и ABI решают разные задачи

УПРОЩЕНИЕ

API: `func add(a, b int) int` — типы, порядок аргументов и результат.

ТОЧНАЯ МОДЕЛЬ

ABI: где физически расположены два `int`, как передать управление и где вернуть результат.

ИНЖЕНЕРНЫЙ ВЫВОД

Совпадение API не означает совпадение машинного соглашения.

Как выглядит вызов на уровне ABI

КОНТРАКТ ВЫЗОВА

```
вызывающая функция
↓ разместить аргументы по ABI
CALL / переход к коду B
↓ вызываемая функция читает аргументы
↓ записывает результат по ABI
возврат вызывающей функции
```

ИНЖЕНЕРНЫЙ ВЫВОД

Вызывающая и вызываемая функции должны независимо соблюдать один и тот же ABI.

Что произойдет при несовместимых ABI

УПРОЩЕНИЕ

Вызывающая функция записала a в регистр RAX, а b — в RBX.

ТОЧНАЯ МОДЕЛЬ

Вызываемая функция ожидает a в стеке, а b в другом регистре. Она прочитает не те данные.

ИНЖЕНЕРНЫЙ ВЫВОД

Корректность отдельной функции недостаточна: нужен общий ABI.

Что входит в соглашение вызова

ЗАДАЧА

Расположение входных значений и результатов, передача управления, адрес возврата, обязанности по сохранению регистров.

МЕХАНИЗМ

Также важны раскладка стека, выравнивание и служебное состояние рантайма.

ИНЖЕНЕРНЫЙ ВЫВОД

ABI охватывает границу между частями кода, а не только список аргументов.

`ABIInternal` — внутренний ABI Go

ЗАДАЧА

Обычные функции Go компилируются под `ABIInternal`: он рассчитан на нужды Go-компилятора и рантайма.

МЕХАНИЗМ

Это не долгосрочный публичный контракт. Регистры и раскладка могут измениться с версией Go.

ИНЖЕНЕРНЫЙ ВЫВОД

Детали `ABIInternal` полезны для диагностики, но не для прикладных гарантий.

ABI0 — граница ассемблер Go

ЗАДАЧА

Ассемблерные функции используют стабильное стековое соглашение ABI0.

МЕХАНИЗМ

Когда обычный Go-код вызывает такую функцию, инструментарий при необходимости строит адаптирующую обертку.

ИНЖЕНЕРНЫЙ ВЫВОД

Два ABI существуют, чтобы совершенствовать Go без постоянной поломки пользовательского asm.

Почему стекового ABI оказалось мало

УПРОЩЕНИЕ

Историческая модель: поместить все аргументы в стек и снова прочитать их в вызываемой функции.

ТОЧНАЯ МОДЕЛЬ

Но значения часто уже находятся в регистрах. Двойное обращение к памяти создает лишние записи и загрузки.

ИНЖЕНЕРНЫЙ ВЫВОД

Передача через регистры уменьшает часть стоимости частых вызовов.

Старый и новый путь передачи значения

МОТИВАЦИЯ

```
значение уже в регистре
↓ стековая схема: STORE → CALL → LOAD
↓ регистровая схема: CALL → использовать регистр
меньше перемещений через память
```

ИНЖЕНЕРНЫЙ ВЫВОД

Регистровый ABI оптимизирует передачу аргументов, не меняя Go API.

Почему регистровый ABI не означает «все в регистрах»

ЗАДАЧА

Регистров меньше, чем потенциальных аргументов; составные типы могут занимать несколько компонентов.

МЕХАНИЗМ

Часть аргументов и результатов по-прежнему идет через стек. И он нужен для локальных значений и spill.

ИНЖЕНЕРНЫЙ ВЫВОД

Современный ABI комбинирует регистры и стек.

Один параметр Go — несколько машинных слов

Рантайм и компилятор уже знают физическое представление строк, срезов, интерфейсов и структур.

```
разложение на компоненты
попытка назначения регистров
иначе весь аргумент в стек
```

Составная сигнатура — хороший стресс-тест ABI

GO / КОМАНДА

```
func f(a int, s string, b []byte,
    x struct { P *Node; N int },
) error
```

ЧТО ПРОИСХОДИТ

Один `string` дает два слова, срез — три, структура — два поля. Логических параметров меньше, чем физических компонентов.

ИНЖЕНЕРНЫЙ ВЫВОД

Количество регистров рассчитывается по представлению типов.

Общий алгоритм размещения аргумента

НАЗНАЧЕНИЕ

```
аргумент T
↓ разложить T на простые компоненты
↓ попробовать назначить всем регистры
поместилось целиком?
├─ да → регистры
└─ нет → весь аргумент в стек
```

ИНЖЕНЕРНЫЙ ВЫВОД

Текущий ABI не обязан делить один логический аргумент между регистрами и стеком.

Почему не разрывать одну структуру пополам

УПРОЩЕНИЕ

`S{A,B,C}`: A и B в регистрах, C в стеке. На первый взгляд, экономим место.

ТОЧНАЯ МОДЕЛЬ

Если нужен `&s`, структуру приходится собирать как единый непрерывный объект памяти. ABI предпочитает целостное размещение.

ИНЖЕНЕРНЫЙ ВЫВОД

Проще отказаться от назначения регистров для всего аргумента.

Целые значения используют отдельные регистры

GO / КОМАНДА

```
func f(a int64, b uint32)
```

ЧТО ПРОИСХОДИТ

Каждый целочисленный компонент может получить собственный аргументный регистр. Маленькие `uint32` не обязаны паковаться попарно.

ИНЖЕНЕРНЫЙ ВЫВОД

Простая схема может быть дешевле постоянных `mask/shift/pack`.

Почему два `uint32` не упаковывают в один регистр

УПРОЩЕНИЕ

Упаковка экономит регистр: два 32-битных аргумента можно разместить в одном 64-битном слове.

ТОЧНАЯ МОДЕЛЬ

Но обеим сторонам вызова нужны упаковка, маски и сдвиги. Текущий ABI предпочитает отдельные компоненты.

ИНЖЕНЕРНЫЙ ВЫВОД

Экономия регистров не всегда окупает стоимость распаковки.

Указателеподобные значения

КОМПОНЕНТЫ

```
*T, unsafe.Pointer
map[K]V, chan T, func(...)
↓ одно указателеподобное слово
↓ один регистр, если есть место
```

ИНЖЕНЕРНЫЙ ВЫВОД

При передаче `map` не копируется вся хеш-таблица: передается ее представление.

`string`: два компонента ABI

STRING

```
Go string
├─ указатель на байты
└─ длина
↓ два независимых машинных слова
↓ два регистра, если хватает
```

ИНЖЕНЕРНЫЙ ВЫВОД

Один параметр `string` может занять два целочисленных регистра.

`[]T`: три компонента ABI

SLICE

```
дескриптор среза
├─ data
├─ len
└─ cap
↓ три машинных компонента
↓ регистры или стек для всего аргумента
```

ИНЖЕНЕРНЫЙ ВЫВОД

Модель среза из основной лекции проявляется прямо в соглашении о вызове.

Интерфейс: два компонента ABI

INTERFACE

```
пустой интерфейс → Type + Data
непустой интерфейс → ITab + Data
↓ два указателеподобных слова
↓ возможны два регистра
```

ИНЖЕНЕРНЫЙ ВЫВОД

Физическая форма интерфейса из 4.1 влияет на передачу через границу функции.

Структура разлагается рекурсивно

GO / КОМАНДА

```
type S struct {
    A int64
    B *Node
}
```

ЧТО ПРОИСХОДИТ

При достаточном числе регистров поле A может пройти через один регистр, а B — через другой.

ИНЖЕНЕРНЫЙ ВЫВОД

Передача структуры по значению не всегда равна большой копии в стеке.

Нулевой и однокомпонентный массивы — особые случаи

ЗАДАЧА

`[0]T` не требует передачи элементов. `[1]T` допускает рекурсивное разложение единственного элемента.

МЕХАНИЗМ

Для массивов длиной больше единицы текущий ABI не использует обычное разложение по регистрам.

ИНЖЕНЕРНЫЙ ВЫВОД

Размер массива — часть правил назначения аргумента.

Почему `[2]T` идет через стек

GO / КОМАНДА

```
func one(x [1]uintptr) uintptr
func two(x [2]uintptr) uintptr
```

ЧТО ПРОИСХОДИТ

Набор регистров нельзя индексировать вычисляемым `i` так же, как непрерывный массив в памяти. Текущий ABI отправляет такой массив в стек.

ИНЖЕНЕРНЫЙ ВЫВОД

Разница между `[1]T` и `[2]T` видна в сгенерированном коде.

Структура и массив с одинаковыми размерами

УПРОЩЕНИЕ

`struct{A, B int}` может быть разложена на два регистра.

ТОЧНАЯ МОДЕЛЬ

`[2]int` в текущем ABI остается стековым аргументом, хотя размер тот же.

ИНЖЕНЕРНЫЙ ВЫВОД

ABI учитывает структуру типа и операции над ним, а не только `sizeof(T)`.

Когда закончились регистры

ПЕРЕПОЛНЕНИЕ

```
тип требует 6 компонентов
↓ осталось 5 доступных регистров
↓ попытка назначения отменяется
↓ весь аргумент — стековая область
```

ИНЖЕНЕРНЫЙ ВЫВОД

Текущий алгоритм использует принцип целостного назначения.

Результаты используют похожее правило

ЗАДАЧА

ABI распределяет возвращаемые значения по регистрам или стеку с отдельным учетом ресурсов для результатов.

МЕХАНИЗМ

Физические регистры могут сначала хранить входные значения, а к возврату — выходные.

ИНЖЕНЕРНЫЙ ВЫВОД

Граница вызова разделяет назначение аргументов и результатов.

Результат `(int, string)` — три компонента

GO / КОМАНДА

```
func f() (int, string)
```

ЧТО ПРОИСХОДИТ

Результаты: `int`, `string.data`, `string.len`. При подходящем бюджете они передаются через регистры.

ИНЖЕНЕРНЫЙ ВЫВОД

Возврат нескольких значений не обязан создавать временную область в хипе.

Большой результат ≠ обязательная аллокация в хипе

УПРОЩЕНИЕ

`func f() BigStruct` выглядит как создание большого объекта и возвращение его целиком.

ТОЧНАЯ МОДЕЛЬ

ABI может предусмотреть стековую область результата. Это не решение эскейп-анализа о хипе.

ИНЖЕНЕРНЫЙ ВЫВОД

Стековая передача результата и аллокация в хипе — разные механизмы.

Стековая раскладка вызова

СТЕК

```
стековые аргументы
↓ выравнивание
стековые результаты
↓ выравнивание
область сохранения регистровых аргументов
```

ИНЖЕНЕРНЫЙ ВЫВОД

Даже у регистрового ABI остается строго определенная память вызова.

Почему регистровому ABI все равно нужен стек

Регистровые аргументы должны переживать рост стека и взятие адреса.

```
область сохранения аргументов (spill)
рост стека
стековая область результата
```

Место сохранения аргументов резервируют заранее

ЗАДАЧА

Аргумент может проходить через регистр, но вызывающая функция все равно резервирует пространство для его сохранения.

МЕХАНИЗМ

В обычном вызове это место может остаться неинициализированным; оно существует для специальных ситуаций.

ИНЖЕНЕРНЫЙ ВЫВОД

Наличие стекового слота не означает, что каждая операция вызывает STORE/LOAD.

Рост стека превращает регистры в сохраняемое состояние

РОСТ СТЕКА

```
вызываемая функция вошла в код
↓ проверка границ стека
↓ свободного места не хватает
↓ сохранить аргументы
↓ увеличить стек и восстановить состояние
```

ИНЖЕНЕРНЫЙ ВЫВОД

Нужно сохранить не только стек, но и аргументы, которые были в регистрах.

Почему это связано с `g0` и `morestack`

ЗАДАЧА

В лекции `Runtime deep` уже разобраны растущие стеки горутин и служебный путь через `g0`.

МЕХАНИЗМ

ABI обязан обеспечить согласованный перенос аргументов через эту границу. Алгоритм роста стека повторно не читаем.

ИНЖЕНЕРНЫЙ ВЫВОД

Соглашение о вызове спроектировано вместе с рантаймом.

Взятие адреса аргумента требует памяти

GO / КОМАНДА

```
func f(x int) {
    p := &x
    use(p)
}
```

ЧТО ПРОИСХОДИТ

Вычислить &x для одного регистра невозможно. При необходимости компилятор материализует доступную по адресу копию значения.

ИНЖЕНЕРНЫЙ ВЫВОД

Регистровый аргумент может попасть в стек без перехода в хип.

Выгрузка в стек и эскейп-анализ — разные механизмы

УПРОЩЕНИЕ

Значение временно выгрузили в стек, потому что нужен адрес или не хватает регистров.

ТОЧНАЯ МОДЕЛЬ

Значение удерживается в хипе, потому что его время жизни или использование требуют этого по результату эскейп-анализа.

ИНЖЕНЕРНЫЙ ВЫВОД

Дополнительные обращения к стеку не доказывают, что объект размещен в хипе.

Почему стек не исчез после смены ABI

ЗАДАЧА

Локальные данные, адресуемые значения, большие аргументы, результаты и временные spill-слоты по-прежнему нужны.

МЕХАНИЗМ

Регистры ускоряют частый перенос через вызов, но не заменяют адресуемую память.

ИНЖЕНЕРНЫЙ ВЫВОД

Правильная модель — регистры как предпочтительный транспорт.

Один ABI-дизайн, разные физические регистры

Алгоритм разложения типов общий; конкретное назначение зависит от ISA.

```
amd64: 9 целочисленных регистров
arm64: 16 целочисленных регистров
специальные регистры рантайм
```

amd64: целочисленные регистры ABIInternal

AMD64

```
RAX → RBX → RCX
→ RDI → RSI
→ R8 → R9 → R10 → R11
всего 9 регистров аргументов/результатов
```

ИНЖЕНЕРНЫЙ ВЫВОД

Точный список относится к текущему внутреннему ABI и может изменяться.

amd64: вещественные аргументы

ЗАДАЧА

Текущий `ABIInternal` использует регистры семейства XMM для вещественных компонентов.

МЕХАНИЗМ

В источнике лекции для них указан диапазон `X0—X14`; это тоже внутренняя деталь ABI.

ИНЖЕНЕРНЫЙ ВЫВОД

Бюджеты целочисленных и вещественных компонентов различаются.

arm64: свой физический набор

ARM64

```
целочисленные аргументы / результаты
R0–R15
↓
вещественные аргументы / результаты
F0–F15
```

ИНЖЕНЕРНЫЙ ВЫВОД

Одинаковая сигнатура Go может получить другое размещение на arm64.

Почему больше регистров не значит автоматически быстрее

УПРОЩЕНИЕ

arm64 имеет больше доступных целочисленных регистров для вызова, чем amd64.

ТОЧНАЯ МОДЕЛЬ

Стоимость зависит от компилятора, давления на регистры, ISA, микроархитектуры и типичных сигнатур.

ИНЖЕНЕРНЫЙ ВЫВОД

Число регистров — только один фактор производительности.

Почему Go не обязан использовать C ABI для своих функций

ЗАДАЧА

На Linux amd64 существует System V ABI, но внутренние функции Go вызываются по собственному соглашению.

МЕХАНИЗМ

Go инструментарий учитывает растущий стек, указатель на G, замыкания и сборщик мусора. При вызове C требуется отдельная адаптация.

ИНЖЕНЕРНЫЙ ВЫВОД

Системный ABI и `ABIInternal` решают разные границы.

Специальные регистры на amd64

ОСОБЫЕ РОЛИ

```
RSP → указатель стека
RBP → указатель кадра
RDX → контекст замыкания
R14 → текущая G
```

ИНЖЕНЕРНЫЙ ВЫВОД

Регистры отличаются по роли; планировщик регистров не может свободно использовать каждый из них.

Почему g удобно держать в регистре

ЗАДАЧА

Рантайм регулярно обращается к состоянию текущей горутины — для стека, планирования и других служебных операций.

МЕХАНИЗМ

На amd64 R14 дает быстрый доступ без явного аргумента на каждом вызове.

ИНЖЕНЕРНЫЙ ВЫВОД

ABI и рантайм связаны скрытым, но строгим контрактом.

Где CPU хранит адрес возврата

УПРОЩЕНИЕ

amd64: `CALL` сохраняет адрес возврата в стековой памяти.

ТОЧНАЯ МОДЕЛЬ

arm64: для адреса возврата используется регистр связи `R30`. При необходимости он сохраняется в кадре функции.

ИНЖЕНЕРНЫЙ ВЫВОД

Даже базовая механика вызова не одинакова на разных ISA.

Как один вызов выглядит на двух архитектурах

СРАВНЕНИЕ

```
одна Go-сигнатура
↓ общий разбор ABI-компонентов
├─ amd64: свой набор регистров / стек
└─ arm64: свой набор регистров / стек
↓ архитектурно-зависимые инструкции
```

ИНЖЕНЕРНЫЙ ВЫВОД

Инженерная модель ABI устойчива; номера регистров — нет.

Замыкание передает больше, чем аргументы

Значение функции должно не только указывать на код, но и обеспечивать доступ к захваченным данным.

```
точка входа + окружение
специальный регистр контекста
обертка метода и ABI
```

Откуда замыкание берет x

GO / КОМАНДА

```
func makeAdder(x int) func(int)
int {
    return func(y int) int {
        return x + y
    }
}
```

ЧТО ПРОИСХОДИТ

Позже вызывающая функция передает только y. Захваченное x должно находиться в объекте окружения замыкания.

ИНЖЕНЕРНЫЙ ВЫВОД

Func значение — больше, чем голый указатель на инструкцию.

Из чего состоит представление значения функции

ЗАМЫКАНИЕ

```
объект замыкания
├─ адрес входа в код
└─ сохраненные захваченные данные
↓ передать адрес окружения
↓ код читает `x` и аргумент `y`
```

ИНЖЕНЕРНЫЙ ВЫВОД

Для функции с окружением нужен дополнительный скрытый контекст.

Специальный регистр контекста замыкания

ЗАДАЧА

Для контекста используется выделенный регистр: в текущей лекции amd64 — RDX, arm64 — R26.

МЕХАНИЗМ

Обычные параметры при этом продолжают назначаться по стандартным правилам ABI.

ИНЖЕНЕРНЫЙ ВЫВОД

Окружение замыкания не сдвигает все обычные аргументы.

Почему не делать окружение первым параметром

УПРОЩЕНИЕ

Можно представить скрытую сигнатуру `func(env *Closure, y int) int`.

ТОЧНАЯ МОДЕЛЬ

Выделенный регистр передает окружение независимо от обычного списка аргументов и упрощает вызовы функция values.

ИНЖЕНЕРНЫЙ ВЫВОД

Скрытые контексты ABI отличаются от параметров Go-сигнатуры.

Значение метода тоже захватывает окружение

GO / КОМАНДА

```
f := obj.Method
f(x)
```

ЧТО ПРОИСХОДИТ

Функция `f` должна помнить конкретный receiver `obj`. По механизму это похоже на замыкание с захваченным значением.

ИНЖЕНЕРНЫЙ ВЫВОД

`obj.Method(x)` и `f := obj.Method; f(x)` могут иметь разные физические пути.

Прямой вызов метода и значение метода

СРАВНЕНИЕ

```
obj.Method(x)
→ receiver задан в месте вызова

f := obj.Method
→ создано значение функции с receiver
→ поздний вызов через сохраненный контекст
```

ИНЖЕНЕРНЫЙ ВЫВОД

Упаковка метода в значение функции имеет собственную цену и жизненный цикл.

Адаптация ABIInternal ↔ ABI0

ABI ОБЕРТКА

```
Go вызывающая функция с ABIInternal
↓ обертка
регистры → стековые аргументы ABI0
↓ вызов asm-функции
стековый результат → регистры ABIInternal
↓ возврат
```

ИНЖЕНЕРНЫЙ ВЫВОД

Обертка реально перекладывает данные между двумя соглашениями.

Когда обертка становится заметной

ЗАДАЧА

Дополнительные перемещения и вызов могут проявиться на границе горячей Go/asm функции.

МЕХАНИЗМ

Но сначала нужен профиль. Сам факт наличия обертки не делает путь проблемой.

ИНЖЕНЕРНЫЙ ВЫВОД

Цена одной границы становится существенной только после умножения на частоту.

Публичная стабильность и свобода компилятора

УПРОЩЕНИЕ

`ABI0` сохраняет контракт пользовательского Go-ассемблера.

ТОЧНАЯ МОДЕЛЬ

`ABIInternal` может развиваться вместе с компилятором и рантаймом: конкретные регистры и раскладка не обещаны навсегда.

ИНЖЕНЕРНЫЙ ВЫВОД

Стабильная граница совместимости сохраняет свободу оптимизаций инструментария.

Итог блока ABI

ОСНОВНАЯ МОДЕЛЬ

```
API: типы и сигнатура
↓ ABIInternal: компоненты / регистры / стек
↓ скрытый контекст при замыкании
↓ обертка ABI0 при необходимости
↓ вызов и результат
```

ИНЖЕНЕРНЫЙ ВЫВОД

ABI соединяет машинные функции и объясняет цену передачи значений.

Как выполнять код, если T неизвестен в исходнике

Для компилятора T — не декоративная буква. Нужно знать размер, указатели и допустимые операции.

```
полная специализация
полностью динамический код
компромисс Go: форма + словарь
```

В чем проблема `Identity[T]`

GO / КОМАНДА

```
func Identity[T any](x T) T {
    return x
}
```

ЧТО ПРОИСХОДИТ

На уровне языка функция проста. Но при создании машинного тела нужно знать, как передать, скопировать и вернуть значение типа T.

ИНЖЕНЕРНЫЙ ВЫВОД

Система типов и генерация машинного кода — разные задачи.

Стратегия 1: отдельная копия для каждого типа

УПРОЩЕНИЕ

`Identity[int]`, `Identity[string]`, `Identity[User]` получают независимые машинные тела.

ТОЧНАЯ МОДЕЛЬ

Максимальная специализация может увеличить бинарник и время компиляции при большом числе типов.

ИНЖЕНЕРНЫЙ ВЫВОД

Полная мономорфизация ускоряет не бесплатно.

Стратегия 2: одна полностью динамическая функция

УПРОЩЕНИЕ

Одна функция принимает описание типа и адрес значения. Исполняемого кода мало.

ТОЧНАЯ МОДЕЛЬ

Размер и операции требуют динамических проверок/загрузок; компилятор заранее знает меньше.

ИНЖЕНЕРНЫЙ ВЫВОД

Экономия машинного кода может увеличить стоимость каждого вызова.

Компромисс текущего Go

ОБЩАЯ СХЕМА

```
параметр типа `T`
↓ группировка совместимых представлений
форма / общее тело
+
словарь конкретной инстанциации
↓ при необходимости обертка
```

ИНЖЕНЕРНЫЙ ВЫВОД

Go сочетает разделяемый код и передачу конкретной информации о типе.

Инстанциация начинается с проверки типов

GO / КОМАНДА

```
Identity(42)
// выведено: T = int
```

ЧТО ПРОИСХОДИТ

Проверка типов определяет конкретный `T=int` и подтверждает допустимость вызова.

ИНЖЕНЕРНЫЙ ВЫВОД

Результат проверки типов еще не означает отдельное машинное тело под `int`.

Инстанциация и генерация кода — два разных решения

ЗАДАЧА

На уровне Go существует конкретная инстанциация `Identity[int]`.

МЕХАНИЗМ

Компилятор может выбрать разделяемую форма-реализацию и передать ей сведения о конкретном `int`.

ИНЖЕНЕРНЫЙ ВЫВОД

Система типов гарантирует семантику, но не число копий кода.

Что компилятор должен знать о T

ЗАДАЧА

Размер и выравнивание, карта Go-указателей, разрешенные операции и способ передачи через ABI.

МЕХАНИЗМ

Для приведения к интерфейсу и вызова методов понадобится еще конкретная идентичность типа.

ИНЖЕНЕРНЫЙ ВЫВОД

Представление значения и конкретная идентичность типа — разные виды информации.

Нельзя автоматически считать «обобщения медленными»

УПРОЩЕНИЕ

Обобщение выглядит как дополнительный слой, значит каждая операция будто бы динамическая.

ТОЧНАЯ МОДЕЛЬ

Компилятор способен специализировать, инлайнить и убрать часть скрытых загрузок. Итог зависит от конкретной инстанциации.

ИНЖЕНЕРНЫЙ ВЫВОД

Сравниваем машинный путь, а не наличие `[T any]` в коде.

Одна форма представления — общий код

Форма типа (shape) позволяет нескольким типам использовать совместимую машинную реализацию.

`go.shape`
создание тела по форме (stenciling)
совместимость представлений ≠ sizeof(T)

Что такое shape

ЗАДАЧА

Компилятор Go использует внутренние типы формы, обозначаемые именами `go.shape...`.

МЕХАНИЗМ

Они описывают группы близких представлений, для которых можно сгенерировать общий код.

ИНЖЕНЕРНЫЙ ВЫВОД

Форма типа (shape) — понятие генератора кода, а не новый тип языка Go.

Одинаковый размер еще не означает одну форму

УПРОЩЕНИЕ

У двух типов может быть одинаковый `sizeof(T)`, например по восемь байт.

ТОЧНАЯ МОДЕЛЬ

Этого недостаточно, чтобы автоматически гарантировать общую форму компиляции; важны и ограничения представления.

ИНЖЕНЕРНЫЙ ВЫВОД

Форма типа не определяется одним лишь количеством байт.

Почему `Zero[T]` подходит для общего тела

GO / КОМАНДА

```
func Zero[T any]() T {
    var z T
    return z
}
```

ЧТО ПРОИСХОДИТ

Если несколько инстанциаций совместимы по форме, компилятор может переиспользовать реализацию вместо полного дублирования.

ИНЖЕНЕРНЫЙ ВЫВОД

Разделение кода снижает рост text-сегмента.

От конкретного типа к реализации по форме

ГЕНЕРАЦИЯ

```
вызовы с T1, T2, T3
↓ анализ совместимости форм
форма S
↓ одна сгенерированная функция
+ словарь для каждой инстанциации
```

ИНЖЕНЕРНЫЙ ВЫВОД

Общее тело для формы сокращает количество кода, но не заменяет информацию о конкретном типе.

Stenciling — создание исполняемого варианта

ЗАДАЧА

Компилятор формирует тело обобщенной функции для выбранной формы или контекста инстанциации.

МЕХАНИЗМ

Это не обязательно отдельная копия для каждого аргумента типа и не одна универсальная функция на все случаи.

ИНЖЕНЕРНЫЙ ВЫВОД

Stenciling — конкретная стратегия генерации кода, зависящая от версии.

Почему общего представления типа недостаточно

УПРОЩЕНИЕ

Форма (shape) определяет, как работать с совместимым представлением значения.

ТОЧНАЯ МОДЕЛЬ

Но `any(x)` требует описатель точного типа `T`, а не только форму, совместимую с несколькими типами.

ИНЖЕНЕРНЫЙ ВЫВОД

Сведения о размещении и идентичность типа нельзя смешивать.

Приведение T к any раскрывает проблему

GO / КОМАНДА

```
func Box[T any](x T) any {
    return x
}
```

ЧТО ПРОИСХОДИТ

Интерфейс содержит Type + Data. Слово Type обязано описывать реальный T конкретного вызова.

ИНЖЕНЕРНЫЙ ВЫВОД

Нужна дополнительная информация, не хранящаяся в одном форма.

Почему тип важен даже при одинаковом представлении

ЗАДАЧА

Два значения могут занимать одинаковое количество машинных слов.

МЕХАНИЗМ

Но рефлексия, утверждение типа и интерфейсная таблица должны различать реальные типы.

ИНЖЕНЕРНЫЙ ВЫВОД

Общий машинный код не отменяет типовую идентичность.

От форма к словарю

ПОЯВЛЕНИЕ СЛОВАРЯ

```
форма → как хранить и копировать
↓
конкретный `T` → какой это тип
↓
словарь инстанциации
↓
общее тело + точные метаданные
```

ИНЖЕНЕРНЫЙ ВЫВОД

Словарь закрывает недостающий слой конкретных типов.

Где хранится конкретная информация о T

Общая форма кода получает метаданные для текущей инстанциации из скрытого словаря.

```
описатели типов / itabs
вложенные словари
указатели на методы
```

Словарь — не просто числовой идентификатор типа

ЗАДАЧА

Для `F[int]` и `F[MyType]` могут существовать разные наборы метаданных.

МЕХАНИЗМ

Словарь содержит ссылки, необходимые общему телу: описатели типов, `itab`, вложенные словари и методы.

ИНЖЕНЕРНЫЙ ВЫВОД

Это таблица конкретных зависимостей инстанциации.

Общая функция и два словаря

СОВМЕСТНОЕ ИСПОЛЬЗОВАНИЕ

```
форма S → общее тело F[S]
├─ словарь для `int`
└─ словарь для `MyType`
↓ тело одно
↓ конкретные описатели разные
```

ИНЖЕНЕРНЫЙ ВЫВОД

Переиспользуем инструкции, не теряя информацию о реальном типе.

Описание типа для any(x)

ЗАДАЧА

При построении интерфейса рантайм-слово Type должно указывать на точный T.

МЕХАНИЗМ

Сгенерированное общее тело может загрузить указатель на соответствующий описатель из словаря.

ИНЖЕНЕРНЫЙ ВЫВОД

Словарь соединяет обобщения с интерфейсным ABI из 4.1.

Инстанциация Box[T]

ПРИМЕР

```
Box[int](x)
↓ выбрать форму
↓ получить словарь `Box[int]`
↓ загрузить тип рантайма для `int`
↓ построить Type + Data
↓ вернуть `any`
```

ИНЖЕНЕРНЫЙ ВЫВОД

Преобразование к интерфейсу требует конкретного типа даже в общем машинном теле.

Почему словарю может понадобиться `itab`

ЗАДАЧА

Возможна операция, где конкретный `T` должен удовлетворять заданному интерфейсу.

МЕХАНИЗМ

Для динамического вызова или приведения может быть подготовлена связь `T ↔ interface`.

ИНЖЕНЕРНЫЙ ВЫВОД

Готовый `itab` в словаре сокращает часть рантайм-поиска.

Словарь и преобразование к интерфейсу

ITAB

```
T + требуемый интерфейс
↓ конкретная таблица методов
↓ слот `itab` в словаре
↓ общее тело получает нужную ссылку
```

ИНЖЕНЕРНЫЙ ВЫВОД

Метаданные интерфейса зависят от реального T, а не только от его формы.

Вложенный вызов обобщенной функции

GO / КОМАНДА

```
func Outer[T any](x T) {
    Inner[T](x)
}
```

ЧТО ПРОИСХОДИТ

Тело `Inner[T]` тоже может ожидать свой словарь. Текущий код может получить вложенную ссылку из словаря `Outer[T]`.

ИНЖЕНЕРНЫЙ ВЫВОД

Словарь способен ссылаться на другой словарь.

Вложенные словари

ВЛОЖЕННЫЙ ВЫЗОВ

```
словарь Outer[T]
↓ ссылка на Inner[T]
↓ словарь Inner[T]
↓ сгенерированное тело Inner[shape]
```

ИНЖЕНЕРНЫЙ ВЫВОД

Обобщенные вызовы образуют граф зависимостей не только кода, но и метаданных.

Выражения методов в словаре

ЗАДАЧА

Если реализация зависит от конкретного метода типа T, форма не всегда определяет нужную функцию.

МЕХАНИЗМ

Словарь может передавать ссылку на выражение метода для этой инстанциации.

ИНЖЕНЕРНЫЙ ВЫВОД

Форма определяет представление; реализация метода зависит от конкретного типа.

Словарь не равен таблице виртуальных методов

УПРОЩЕНИЕ

Можно подумать, что словарь содержит только методы, как `ITab`.

ТОЧНАЯ МОДЕЛЬ

Он также может хранить типы рантайма, вложенные словари и другие ссылки для инстанциации.

ИНЖЕНЕРНЫЙ ВЫВОД

Словарь обобщения шире одного интерфейсного dispatch.

Что генерирует компилятор при обобщении

РЕЗУЛЬТАТ

```
исходное объявление `F[T]`
↓ форма для общего машинного тела
↓ словари конкретных вызовов
↓ при необходимости обертки
↓ символы кода и данных для компоновщика
```

ИНЖЕНЕРНЫЙ ВЫВОД

Обобщения создают и инструкции, и новый граф метаданных.

Словарь надо физически передать в функцию

Скрытые метаданные обобщения превращаются в часть двоичного вызова.

```
явные аргументы Go
+ словарь инстанциации
→ реальная сигнатура ABI
```

Скрытый параметр обобщенной функции

СКРЫТЫЙ АРГУМЕНТ

```
Go: F[T](x)
↓ выбрать общее тело для форма
↓ получить словарь конкретного T
↓ машинный вызов: (словарь, x)
↓ ABI назначает физические места
```

ИНЖЕНЕРНЫЙ ВЫВОД

Семантическая сигнатура Go может быть короче реальной сгенерированной.

Почему это прямо связано с ABI

УПРОЩЕНИЕ

На уровне Go программист передает только X.

ТОЧНАЯ МОДЕЛЬ

Общему сгенерированному телу дополнительно нужен указатель на словарь. Его размещение тоже определяет ABI.

ИНЖЕНЕРНЫЙ ВЫВОД

Генерация обобщений и соглашение о вызове — один стек механизмов.

Дополнительный указатель тоже расходует ресурс

ЗАДАЧА

Скрытый параметр занимает регистр или стековое место по правилам машинного вызова.

МЕХАНИЗМ

На горячем пути это может добавить движения и загрузки метаданных.

ИНЖЕНЕРНЫЙ ВЫВОД

Плата обобщения проявляется физически, но ее нужно измерять.

Полная цепочка `F[int](x)`

GO / КОМАНДА

```
F[int](x)
```

ЧТО ПРОИСХОДИТ

Компилятор выводит `T=int`, выбирает форма, находит словарь, назначает скрытый и явный аргументы и вызывает сгенерированное тело.

ИНЖЕНЕРНЫЙ ВЫВОД

Обычная строка Go может иметь дополнительный физический параметр.

От места вызова до нужного тела

СКВОЗНОЙ ВЫЗОВ

```
T = int
↓ форма S
↓ словарь[int]
↓ ABI: словарь + x
↓ CALL F[shape]
↓ чтение словарных слотов при необходимости
```

ИНЖЕНЕРНЫЙ ВЫВОД

Форма и словарь соединяют проверку типов с машинным ABI.

Почему накладные расходы могут исчезнуть

ЗАДАЧА

После инлайнинга часть словарных ссылок может стать известной компилятору как константы.

МЕХАНИЗМ

Затем ненужные загрузки и ветви удаляются обычными оптимизациями.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя оценивать стоимость обобщения до просмотра итогового кода.

Типовая специализация и разделение инструкций

УПРОЩЕНИЕ

Отдельная полная функция под каждый T дает максимум статической информации, но увеличивает код.

ТОЧНАЯ МОДЕЛЬ

Общее тело форма экономит код, но нуждается в словаре конкретной инстанциации.

ИНЖЕНЕРНЫЙ ВЫВОД

Цена решения делится между размером бинарника и работой на вызове.

Почему `ITab` не вызывает форма напрямую

Интерфейс ожидает вызываемый метод конкретного типа; общему обобщенному телу нужен словарь.

```
метод конкретной инстанциации
обертка
словарь → общее тело
```

Метод обобщенного типа

GO / КОМАНДА

```
type Box[T any] struct { V T }
func (b Box[T]) Get() T {
    return b.V
}
```

ЧТО ПРОИСХОДИТ

У `Box[int]` есть обычная наблюдаемая сигнатура метода. Внутри его исполнение может вести к общему телу по форма.

ИНЖЕНЕРНЫЙ ВЫВОД

Методная таблица и обобщенный код должны согласовать вызов.

Зачем нужна методная обертка

ОБЕРТКА

```
интерфейсный вызов
↓ ITab.Fun[]
↓ обертка конкретного Box[int]
↓ подставить словарь Box[int]
↓ вызвать Get[shape]
```

ИНЖЕНЕРНЫЙ ВЫВОД

Обертка заранее адаптирует обычный метод к внутренней обобщенной реализации.

Почему `ITab` не ищет словарь при каждом вызове

УПРОЩЕНИЕ

`ITab` должен был бы сам понимать форма, типовые аргументы и набор скрытых метаданных.

ТОЧНАЯ МОДЕЛЬ

Сгенерированная обертка уже знает инстанциацию и передает правильный словарь.

ИНЖЕНЕРНЫЙ ВЫВОД

Сложность перенесена в подготовленный код обертки.

Обертка — не обязательно бесплатный переход

ЗАДАЧА

На горячем пути может остаться дополнительный вызов, указатель на словарь и связанные загрузки.

МЕХАНИЗМ

Но инлайнинг иногда удаляет эту границу. Наличие символа обертки не равно доказанной потере производительности.

ИНЖЕНЕРНЫЙ ВЫВОД

Нужно исследовать фактический путь конкретного метода.

Почему shape затрудняет девиртуализацию

ЗАДАЧА

Компилятор знает абстрактную форму receiver. Но вызову конкретного метода могут требоваться данные определенной инстанциации.

МЕХАНИЗМ

Обычная замена вызова через интерфейс прямым методом становится менее очевидной.

ИНЖЕНЕРНЫЙ ВЫВОД

Ограничение оптимизации вызвано недостающей типовой информацией.

Цепочка возможной цены метода

СТОИМОСТЬ

```
receiver обобщенной формы
↓ нужен конкретный метод T
↓ метаданные / обертка
↓ вызов реализации
↓ инлайнинг может убрать часть слоев
```

ИНЖЕНЕРНЫЙ ВЫВОД

Одна и та же конструкция может иметь разные остаточные расходы после оптимизаций.

Ограничение задает допустимые операции

GO / КОМАНДА

```
func Add[T ~int | ~int64](a, b T)
T {
    return a + b
}
```

ЧТО ПРОИСХОДИТ

Constraint разрешает +. Но физическая ширина операции и форма тела все еще зависят от представления типов.

ИНЖЕНЕРНЫЙ ВЫВОД

Ограничение языка и машинная специализация — разные стадии.

Обобщенные методы в рассматриваемой версии Go

ЗАДАЧА

Исходная лекция рассматривает новые обобщенные методы конкретных типов в Go 1.27.

МЕХАНИЗМ

Ограничение на обобщенные методы самих интерфейсов сохраняется; это влияет на генерацию набора методов.

ИНЖЕНЕРНЫЙ ВЫВОД

Версионно-зависимые возможности нельзя автоматически переносить на старые версии компилятора.

Почему обобщенный метод интерфейса сложнее

УПРОЩЕНИЕ

Обычный интерфейс имеет заранее определенный набор методов и подготовленные адреса в `ITab`.

ТОЧНАЯ МОДЕЛЬ

Параметризованный метод мог бы потребовать новые инстанциации, когда конкретный receiver станет известен только при выполнении.

ИНЖЕНЕРНЫЙ ВЫВОД

Вычислимость требуемого машинного кода становится сложнее.

Полный путь интерфейсного метода

МОСТ К 4.1

```
значение интерфейса
↓ ITab.Fun
↓ обертка конкретной инстанциации
↓ словарь для T
↓ общее тело Get[shape]
↓ результат
```

ИНЖЕНЕРНЫЙ ВЫВОД

Этот путь снова соединяет 4.1, компилятор 4.3 и ABI 4.4.

Чем платим за переиспользование общего кода

Смотрим размер кода, скрытые параметры, словарные загрузки и возможность инлайнинга.

```
специализация ↔ размер
словарь ↔ дополнительная работа
профиль → SSA / objdump
```

Две противоположные стоимости

УПРОЩЕНИЕ

Больше отдельных сгенерированных тел: размер `text` и время компиляции могут вырасти.

ТОЧНАЯ МОДЕЛЬ

Больше общего кода: появляются зависимости от словаря и возможные дополнительные вызовы.

ИНЖЕНЕРНЫЙ ВЫВОД

Генератор балансирует размер кода и стоимость выполнения.

Почему instruction кеш участвует в компромиссе

ЗАДАЧА

Сотни копий больших функций увеличивают рабочий набор машинных инструкций.

МЕХАНИЗМ

При подходящей нагрузке это может привести к промахам кеша инструкций и задержкам frontend CPU.

ИНЖЕНЕРНЫЙ ВЫВОД

Размер бинарника способен отразиться на выполнении, но не обязан.

Почему общий код тоже не бесплатен

ЗАДАЧА

Скрытый аргумент, словарная загрузка, обертка и косвенная ссылка иногда остаются в исполняемом пути.

МЕХАНИЗМ

Оптимизатор может убрать часть этой работы, если видит конкретные значения.

ИНЖЕНЕРНЫЙ ВЫВОД

Накладные расходы надо подтверждать в сгенерированном коде.

«Обобщения всегда медленнее» — неверная модель

УПРОЩЕНИЕ

Обобщения добавляют слой форма/словаря и поэтому выглядят заведомо дорогими.

ТОЧНАЯ МОДЕЛЬ

Реальный код может содержать специализацию, инлайнинг и удаленные словарные обращения.

ИНЖЕНЕРНЫЙ ВЫВОД

Вывод делается только после профилирования и просмотра сборки.

nm показывает реальные символы форм

GO / КОМАНДА

```
go tool nm ./app | grep -E
'shape|dict'
```

ЧТО ПРОИСХОДИТ

Ищем сгенерированные тела, словари и обертки. Не предполагаем заранее ни число копий, ни имена символов.

ИНЖЕНЕРНЫЙ ВЫВОД

Сгенерированные символы — диагностические данные, не публичный API.

Диагностика горячей обобщенной функции

ЭКСПЛУАТАЦИЯ

```
CPU профиль / бенчмарк
↓ найти горячую инстанциацию
↓ go tool nm
↓ objdump / SSA
↓ есть словарные загрузки, обертка, косвенный вызов?
↓ изменить и измерить снова
```

ИНЖЕНЕРНЫЙ ВЫВОД

Обобщение анализируем теми же инженерными методами, что и обычный Go-код.

Граница гарантий обобщения

ЗАДАЧА

Спецификация обещает семантику типов, ограничений и обобщенных функций.

МЕХАНИЗМ

`go.shape`, устройство словарей, число тел, обертки и скрытые параметры относятся к текущему компилятору.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя строить корректность приложения на именах внутренних символ формыов.

Почему готовые функции еще не образуют программу

Компилятор создал код и данные, но адреса и связи между пакетами предстоит собрать.

```
символы
релокации
граф всей программы
```

Компиляция отдельного пакета не дает финальных адресов

УПРОЩЕНИЕ

Пакет B вызывает `A.F()`, но при своей компиляции может не знать адрес `F` в будущем бинарнике.

ТОЧНАЯ МОДЕЛЬ

Вместо абсолютного адреса остается ссылка на символ `A.F`, которую позднее разрешит компоновщик.

ИНЖЕНЕРНЫЙ ВЫВОД

Межпакетные вызовы сначала существуют как символические зависимости.

Что такое символ компоновщика

ЗАДАЧА

Именованный объект кода, данных или метаданных: функция, строка, описатель типа, обертка или словарь.

МЕХАНИЗМ

У него есть имя, содержимое/размер, свойства и ссылки на другие символы.

ИНЖЕНЕРНЫЙ ВЫВОД

Символ — вершина общей графовой модели программы.

Примеры символов Go

ОБЪЕКТЫ КОМПОНОВКИ

```
main.main
runtime.chansend
mypkg.Process
type:*mypkg.User
строковые данные
словарь обобщенной функции
```

ИНЖЕНЕРНЫЙ ВЫВОД

Символы обозначают не только машинные функции.

Что скрывает CALL A.F

GO / КОМАНДА

```
func Run() {
    A.F()
}
```

ЧТО ПРОИСХОДИТ

На этапе компиляции пакета может быть известен символ A.F, но неизвестно его окончательное смещение в исполняемом файле.

ИНЖЕНЕРНЫЙ ВЫВОД

Нужна отложенная привязка вызова к адресу.

Релокация — отложенная адресная ссылка

ЗАДАЧА

Объектный код помечает место, где потребуется адрес или смещение другого символа.

МЕХАНИЗМ

Компоновщик размещает символы, затем вычисляет и применяет нужное значение.

ИНЖЕНЕРНЫЙ ВЫВОД

Релокация — ребро графа плюс инструкция, как его закодировать.

Как применяется релокация вызова

РЕЛОКАЦИЯ

```
машинный код: CALL [адрес неизвестен]
↓ релокация → символ A.F
↓ компоновщик выбирает размещение
↓ становится известен address(A.F)
↓ записать смещение
готовый CALL
```

ИНЖЕНЕРНЫЙ ВЫВОД

После размещения символов ссылка превращается в машинный операнд.

Почему компилятор не может заранее выбрать адрес

УПРОЩЕНИЕ

Каждый пакет знает свои объявления и ссылки.

ТОЧНАЯ МОДЕЛЬ

Окончательный размер, выравнивание, порядок и состав кода зависят от всех подключенных пакетов и рантайма.

ИНЖЕНЕРНЫЙ ВЫВОД

Финальная адресная карта — задача компоновщика.

Релокации нужны не только для вызовов

ЗАДАЧА

Ссылка может указывать на глобальную переменную, строку, описатель типа, словарь или таблицу методов.

МЕХАНИЗМ

Компоновщик связывает и управление, и данные, и служебную информацию рантайма.

ИНЖЕНЕРНЫЙ ВЫВОД

Структура бинарника — граф кода и метаданных.

Программа как граф символов

ГРАФ

```
main.main
├─ app.Run
├─ type:Config
└─ строка "start"
app.Run → runtime.chansend
app.Run → обертка → словарь → тело по форме
```

ИНЖЕНЕРНЫЙ ВЫВОД

> Символы — вершины; релокации и ссылки — ребра.

Загрузчик символов объединяет объектные данные

ЗАДАЧА

Скомпилированные пакеты содержат наборы символов, релокаций и метаданных.

МЕХАНИЗМ

Внутренний загрузчик компоновщика строит общую систему имен и ссылок для всей программы.

ИНЖЕНЕРНЫЙ ВЫВОД

Формат объектных файлов здесь не разбираем — важен единый граф.

Почему не каждая скомпилированная функция попадет в бинарник

Компоновщик исследует, какие символы могут понадобиться при выполнении.

```
корни графа
обход ссылок
удаление недостижимого
```

Функцию можно скомпилировать и выбросить

GO / КОМАНДА

```
func Used() {}
func NeverCalled() {}
```

ЧТО ПРОИСХОДИТ

Если до `NeverCalled` нет допустимого пути от корневых символов программы, компоновщик может не включать ее код.

ИНЖЕНЕРНЫЙ ВЫВОД

Наличие объявления не гарантирует наличие функции в конечном бинарнике.

Алгоритм достижимости символов

DEAD CODE

```
начальные корни: точки входа / init / рантайм
↓ пометить корни
↓ пройти ссылки доступных символов
↓ пометить новые вершины
↓ повторять до завершения
↓ удалить недостижимый код
```

ИНЖЕНЕРНЫЙ ВЫВОД

Это обход графа всей программы, а не проверка одной функции.

Что считается корнями

ЗАДАЧА

Точки входа, обязательные функции рантайма, задачи инициализации и определенные служебные метаданные.

МЕХАНИЗМ

Из этих вершин компоновщик начинает поиск используемой части программы.

ИНЖЕНЕРНЫЙ ВЫВОД

Достижимость отвечает «может понадобиться?», а не «часто выполняется?».

Удаление мертвого кода в компиляторе и компоновщике

УПРОЩЕНИЕ

Компилятор в 4.3 удалял недостижимые базовые блоки и лишние значения внутри функций.

ТОЧНАЯ МОДЕЛЬ

Компоновщик может удалить целую функцию/символ, если она недостижима из графа всей программы.

ИНЖЕНЕРНЫЙ ВЫВОД

Два этапа устраняют разную избыточность.

Импорт пакета не равен копированию всех его функций

ЗАДАЧА

В библиотеке могут быть сотни функций, но приложению нужна малая часть.

МЕХАНИЗМ

Компонентный обход позволяет не включать многие недостижимые символы.

ИНЖЕНЕРНЫЙ ВЫВОД

Состав бинарника определяется ссылками и требованиями рантайма.

Код, который остается, и код, который уходит

ПРИМЕР

```
основной путь → A → B → C
                  └→ D
отдельный F без входящих ссылок
↓
A/B/C/D — достижимы
F — кандидат на удаление
```

ИНЖЕНЕРНЫЙ ВЫВОД

Удаление целых символов уменьшает конечный машинный код.

Горячий код и достижимый код — не одно и то же

УПРОЩЕНИЕ

Редко используемая функция может занимать место в исполняемом файле.

ТОЧНАЯ МОДЕЛЬ

Если она достижима и нужна для корректности, компоновщик должен ее сохранить.

ИНЖЕНЕРНЫЙ ВЫВОД

Анализ профиля и анализ достижимости отвечают на разные вопросы.

Почему компоновщику нужен запас корректности

ЗАДАЧА

Не все вызовы выглядят как прямой CALL с явной ссылкой на конечную функцию.

МЕХАНИЗМ

Интерфейсы, рефлексия и обобщения добавляют косвенные пути.

ИНЖЕНЕРНЫЙ ВЫВОД

Простой обход прямых вызовов мог бы удалить живой код.

Почему прямой граф CALL недостаточен

Метод может вызываться через ITab или быть найден по имени уже при выполнении.

```
интерфейсный метод
служебные ссылки компилятора
консервативное сохранение
```

Вызов метода может быть косвенным

GO / КОМАНДА

```
var w io.Writer = someValue
w.Write(buf)
```

ЧТО ПРОИСХОДИТ

Машинный путь может идти через `ITab.Fun[]`. Прямой `CALL (*MyWriter).Write` не обязан стоять в месте вызова.

ИНЖЕНЕРНЫЙ ВЫВОД

Метод может быть необходим без явного ребра прямого вызова.

Как компоновщик мог бы ошибиться

УПРОЩЕНИЕ

Если искать только `CALL`, тело `T.Write` выглядит неиспользуемым.

ТОЧНАЯ МОДЕЛЬ

Но `ITab` может привести к `T.Write` во время работы программы. Удаление нарушит семантику интерфейсов.

ИНЖЕНЕРНЫЙ ВЫВОД

Нужно учитывать косвенную достижимость методов.

Компилятор оставляет маркеры использования методов

ЗАДАЧА

В объектных данных появляются специальные ссылки/метаданные об интерфейсных типах и вызываемых методах.

МЕХАНИЗМ

Компоновщик учитывает их при удалении неиспользуемых методов.

ИНЖЕНЕРНЫЙ ВЫВОД

Между компилятор и компоновщик существует скрытый контракт о метод reachability.

Достижимость через интерфейс

ITAB

```
вызов `w.Write`
↓ компилятор метаданные: нужен метод Write
↓ concrete type может реализовать интерфейс
↓ сохранить соответствующую реализацию
↓ ITab.Fun ссылается на живое тело
```

ИНЖЕНЕРНЫЙ ВЫВОД

Интерфейсный вызов должен остаться корректным после уменьшения бинарника.

Обобщенные формы усложняют сопоставление методов

ЗАДАЧА

При работе с формами конкретная сигнатура или receiver могут быть представлены менее точно.

МЕХАНИЗМ

Компоновщик выбирает более консервативный способ определения потенциально нужных методов.

ИНЖЕНЕРНЫЙ ВЫВОД

Меньше статической информации — шире множество сохраняемых символов.

Рефлексия добавляет вызов метода по имени

GO / КОМАНДА

```
reflect.TypeOf(x).MethodByName("Run")
```

ЧТО ПРОИСХОДИТ

Явного `CALL T.Run` может не быть. Но код во время выполнения способен найти метод через метаданные типа.

ИНЖЕНЕРНЫЙ ВЫВОД

Компоновщик обязан учитывать динамический доступ к методам.

Почему рефлексия может увеличить бинарник

УПРОЩЕНИЕ

Статический граф явно показывает лишь известные вызовы.

ТОЧНАЯ МОДЕЛЬ

Имя метода может вычисляться во время выполнения. Для корректности приходится сохранять часть потенциально вызываемых методов.

ИНЖЕНЕРНЫЙ ВЫВОД

Цена рефлексии проявляется не только в CPU, но и в составе бинарника.

Почему нельзя утверждать, что рефлексия сохраняет все

ЗАДАЧА

Конкретное влияние зависит от видов рефлексия, экспортируемых методов и метаданных, которые доступны анализу.

МЕХАНИЗМ

Компоновщик не обязан включать вообще каждую функцию всей программы.

ИНЖЕНЕРНЫЙ ВЫВОД

Диагноз — через реальные символы и размеры, а не лозунг.

Выбор метода по строке ломает замкнутый список

REFLECTION

```
MethodByName(name)
↓ имя из конфигурации / запроса
↓ компилятор не знает точный метод
↓ компоновщик расширяет допустимое множество
↓ дополнительные методы могут сохраниться
```

ИНЖЕНЕРНЫЙ ВЫВОД

Динамическое имя уменьшает точность статического анализа.

Рефлексия и обычный интерфейс

УПРОЩЕНИЕ

Интерфейс задает статически известный набор методов, хотя конкретный receiver может меняться.

ТОЧНАЯ МОДЕЛЬ

Рефлексия может выбрать метод по имени во время работы программы, поэтому его достижимость труднее доказать.

ИНЖЕНЕРНЫЙ ВЫВОД

Разные формы динамичности требуют разной консервативности.

Обобщения создают граф кода и метаданных

Тела функций по форме, словари, обертки, типы и методы становятся объектами компоновщика.

```
вызывающая функция → словарь → тип / itab
обертка → общее тело
достижимость всего графа
```

Какие символы возникают после обобщения

СОСТАВ

```
тела по форме функций
словарь каждой нужной инстанциации
методные обертки
описатели рантайм-типов
ссылки на itab и другие словари
```

ИНЖЕНЕРНЫЙ ВЫВОД

Нужно компоновать не только код, но и новые зависимые данные.

Словарь — тоже символ данных

ЗАДАЧА

Обобщенный вызов может иметь ссылку на словарь конкретной инстанциации.

МЕХАНИЗМ

Если вызывающая функция достижима и ей нужен словарь, эта зависимость тоже становится достижимой.

ИНЖЕНЕРНЫЙ ВЫВОД

Релокации переносят информацию о типе между этапами компиляции и компоновки.

Достижимость словаря

ПРОХОД

```
достижимый вызов F[int]
↓ релокация к словарю[int]
словарь становится достижимым
↓ ссылки на описатели типов / itab
↓ дополнительные символы сохраняются
```

ИНЖЕНЕРНЫЙ ВЫВОД

Словарь разворачивает еще один слой графа зависимостей.

Почему словарь может удерживать методную обертку

ЗАДАЧА

Внутри метаданных инстанциации могут быть ссылки на конкретные методы.

МЕХАНИЗМ

Если такой слот нужен, соответствующие обертка и тело должны сохраниться в бинарнике.

ИНЖЕНЕРНЫЙ ВЫВОД

Достижимость обобщенного вызова не ограничивается одним CALL к телу по форме.

`Box[int]` требует описатель типа

GO / КОМАНДА

```
type Box[T any] struct { V T }
var x any = Box[int]{V: 7}
```

ЧТО ПРОИСХОДИТ

Для интерфейсов и рефлексия рантайму понадобится информация о конкретном `Box[int]`: размер, указатели и методы.

ИНЖЕНЕРНЫЙ ВЫВОД

Инстанциация порождает метаданные, доступные компоновщику.

Почему метаданные типов необходимы рантайму

ЗАДАЧА

Описатели типов используются интерфейсами, рефлексией, GC и сравнениями типов.

МЕХАНИЗМ

Поэтому это не просто информация для отладчика, которую можно безнаказанно удалить.

ИНЖЕНЕРНЫЙ ВЫВОД

Go-бинарник содержит обязательную для исполнения метаинформацию.

Обобщенный вызов после компоновки

СКВОЗНОЙ ПУТЬ

```
F[int](x) в Go
↓ форма + словарь[int]
↓ ABI назначает скрытый аргумент
↓ символы и релокации
↓ компоновщик оставляет достижимое
↓ исполняемый вызов
```

ИНЖЕНЕРНЫЙ ВЫВОД

Все три большие части лекции соединяются на одной операции.

От символических ссылок к адресам и стековым трассам

После отбора символов компоновщик создает адресную карту и служебные таблицы рантайма.

```
размещение → релокации
pclntab
PC → функция → файл:строка
```

Зачем нужен этап размещения

LAYOUT

```
достижимые символы кода и данных
↓ сгруппировать по назначению
код / константы / данные / метаданные
↓ выравнивание и адреса
↓ применить релокации
```

ИНЖЕНЕРНЫЙ ВЫВОД

Только после размещения можно окончательно закодировать многие межсимвольные ссылки.

Релокация зависит от архитектуры

ЗАДАЧА

На amd64 адрес вызова может кодироваться относительным смещением; arm64 имеет другие ограничения и формы инструкций.

МЕХАНИЗМ

Поэтому компоновщик учитывает ISA, а не только имена символов.

ИНЖЕНЕРНЫЙ ВЫВОД

Значение релокации — часть машинного формата.

Что если переход слишком далеко

ЗАДАЧА

У некоторых инструкций диапазон непосредственного перехода ограничен.

МЕХАНИЗМ

Компоновщику может понадобиться промежуточный переход или специальная последовательность инструкций.

ИНЖЕНЕРНЫЙ ВЫВОД

Связывание — не всегда простая подстановка целого адреса.

Почему исполняемый файл не состоит из одного `.text`

УПРОЩЕНИЕ

Можно представлять бинарник как большой массив машинных инструкций.

ТОЧНАЯ МОДЕЛЬ

Реальная программа содержит код, константы, изменяемые данные, описатели типов, GC-таблицы и сведения о функциях.

ИНЖЕНЕРНЫЙ ВЫВОД

Формат исполняемого файла разделяет код, данные и метаданные.

Рантайм видит адрес, но хочет имя функции

GO / КОМАНДА

```
PC = 0x12345678
// ожидаем: mypkg.Process /
process.go:142
```

ЧТО ПРОИСХОДИТ

CPU возвращает числовой адрес. Для трассировки стека Go должен сопоставить его с функцией и строкой исходника.

ИНЖЕНЕРНЫЙ ВЫВОД

Нужна таблица связей между адресами и исходником.

`pclntab` — данные рантайма, а не просто debug-файл

ЗАДАЧА

Компоновщик строит таблицы PC и функций с именами, файлами, а также дополнительными `pcdata/funcdata`.

МЕХАНИЗМ

Сам рантайм Go использует эти таблицы для поиска функции и исходной строки по адресу.

ИНЖЕНЕРНЫЙ ВЫВОД

Трассировка Go возможна без внешнего отладчика.

Как адрес превращается в читаемую трассировку стека

СИМВОЛИЗАЦИЯ

```
PC из стека
↓ найти диапазон функции
↓ метаданные `pclntab`
↓ имя функции
↓ файл и строка
↓ читаемый кадр стека
```

ИНЖЕНЕРНЫЙ ВЫВОД

> `pclntab` делает адреса CPU понятными разработчику.

`runtime.Callers` работает с PC

GO / КОМАНДА

```
pcs := make([]uintptr, 16)
n := runtime.Callers(0, pcs)
```

ЧТО ПРОИСХОДИТ

Функция собирает адреса кадров. Следующий этап через метаданные восстанавливает имена функций и исходные строки.

ИНЖЕНЕРНЫЙ ВЫВОД

`Callers` и `CallersFrames` демонстрируют использование `pclntab`.

Почему pprof тоже зависит от инструментария

ЗАДАЧА

CPU-профиль начинает с адресов исполнения и стеков вызовов.

МЕХАНИЗМ

Чтобы получить `mypkg.Handler` вместо `0x...`, нужны функция/строка из сгенерированных метаданных.

ИНЖЕНЕРНЫЙ ВЫВОД

Наблюдаемость — совместный результат работы компилятора, компоновщика и рантайма.

Метаданные рантайма, DWARF и режимы компоновки

Не все таблицы в бинарнике нужны одной и той же подсистеме.

```
pclntab ≠ DWARF
-s / -w
внутренняя / внешняя компоновка
```

`pclntab` и DWARF — разные системы

УПРОЩЕНИЕ

`pclntab` нужен самому рантайму Go для перехода от PC к функциям и строкам.

ТОЧНАЯ МОДЕЛЬ

DWARF рассчитан на внешние отладчики: типы, переменные, области видимости, места хранения и инлайнинг.

ИНЖЕНЕРНЫЙ ВЫВОД

Удаление внешней отладочной информации не обязано ломать трассировку panic.

Почему оптимизированный код трудно дебажить

ЗАДАЧА

Переменная могла исчезнуть, стать константой, жить в регистре или быть перемещенной после инлайнинга.

МЕХАНИЗМ

DWARF должен описать связь исходного кода с уже преобразованными инструкциями.

ИНЖЕНЕРНЫЙ ВЫВОД

Отладчик восстанавливает исходную модель из измененного машинного мира.

Что делают `-S` и `-W`

GO / КОМАНДА

```
go build -ldflags='-s -w' -o app
./cmd/app
```

ЧТО ПРОИСХОДИТ

`-W` убирает DWARF; `-S` сокращает таблицы символов и отладочную информацию. Метаданные, необходимые рантайму, существуют отдельно.

ИНЖЕНЕРНЫЙ ВЫВОД

Удаление отладочных символов уменьшает размер файла, но не удаляет всю метаинформацию Go.

Почему трассировка panic живет после удаления символов

УПРОЩЕНИЕ

Ошибочная модель: удалили символы — рантайм больше не узнает имена функций.

ТОЧНАЯ МОДЕЛЬ

Go сохраняет необходимые самому рантайму таблицы, включая данные для трассировки.

ИНЖЕНЕРНЫЙ ВЫВОД

Нужно различать информацию внешнего отладчика и внутренние таблицы.

Внутренняя компоновка

ЗАДАЧА

Для простого проекта на чистом Go собственный компоновщик может связать код и данные и создать исполняемый файл для нужной платформы.

МЕХАНИЗМ

Он знает Go-объекты, релокации и обязательные таблицы рантайма.

ИНЖЕНЕРНЫЙ ВЫВОД

Внешний системный компоновщик требуется не для каждой Go-программы.

Внешняя компоновка

ЗАДАЧА

При интеграции с C и платформенными объектами может понадобиться системный компоновщик.

МЕХАНИЗМ

Go готовит промежуточные данные и передает часть завершающей работы внешней программе.

ИНЖЕНЕРНЫЙ ВЫВОД

Режим линковки зависит от окружения сборки, а не только от исходных Go-функций.

Что меняет cgo

ABI LINKER

```
Go функция / ABIInternal
↓ граница C ABI
платформенные объектные файлы
↓ внешние библиотеки / релокации
↓ возможно внешний компоновщик
```

ИНЖЕНЕРНЫЙ ВЫВОД

cgo приносит второй двоичный мир и новые зависимости.

Что такое готовый исполняемый файл

ЗАДАЧА

На Linux — ELF, на macOS — Mach-O, на Windows — PE.

МЕХАНИЗМ

Внутри находятся заголовки, код, разные классы данных, метаданные рантайма и, возможно, отладочные разделы.

ИНЖЕНЕРНЫЙ ВЫВОД

Файл исполняемой программы структурирован; его загрузка — тема 4.5.

Почему 4.4 заканчивается до загрузчика ОС

ЗАДАЧА

Компоновщик создает исполняемый файл и финальную адресную карту на своем уровне.

МЕХАНИЗМ

ОС позже интерпретирует формат, отображает сегменты и передает управление точке входа.

ИНЖЕНЕРНЫЙ ВЫВОД

Компоновка и загрузка процесса — разные технические слои.

Собираем все три слоя в один исполняемый путь

Обычный и обобщенный вызов проходят через ABI, символы и компоновщик.

```
вызов → регистры / стек
форма → словарь / обертка
символы → ссылки → адреса
```

Обычный вызов от Go до CPU

ОБЫЧНАЯ ФУНКЦИЯ

```
r := F(a, b)
↓ компилятор → машинный символ функции
↓ ABI: разместить a и b
↓ релокация вызова F
↓ компоновщик назначил адрес F
↓ CALL → получить результат
```

ИНЖЕНЕРНЫЙ ВЫВОД

Физическое исполнение требует и ABI, и связанного адреса функции.

Обобщенный вызов от Go до CPU

ОБОБЩЕНИЯ

```
F[int](x)
↓ инстанциация T=int
↓ форма + словарь[int]
↓ ABI: скрытый словарь + x
↓ CALL к символу общего тела
↓ компоновщик связал код и данные
```

ИНЖЕНЕРНЫЙ ВЫВОД

> Даже один обобщенный вызов затрагивает все три темы.

Интерфейсный метод обобщенного типа

ITAB + ОБЕРТКА

```
значение интерфейса
↓ ITab.Fun
↓ методная обертка Box[int]
↓ словарь конкретного T
↓ общее тело метода для shape
↓ вернуть результат через ABI
```

ИНЖЕНЕРНЫЙ ВЫВОД

Обертка адаптирует ожидаемую сигнатуру метода к обобщенной реализации.

Замыкание тоже проходит через компоновку

ЗАМЫКАНИЕ

```
f := makeAdder(10)
↓ объект: адрес кода + захваченное x
↓ вызов f(5)
↓ регистр контекста + аргумент
↓ адрес кода связан компоновщиком
```

ИНЖЕНЕРНЫЙ ВЫВОД

Контекст замыкания и конечный адрес функции — два независимых условия корректного вызова.

Как Go строит трассировку стека

НАБЛЮДАЕМОСТЬ

```
выполняемый машинный код
↓ PC и адреса возврата
↓ обход стека
↓ pclntab / данные функций
↓ имя функции, файл и строка
```

ИНЖЕНЕРНЫЙ ВЫВОД

Трассировка стека зависит от ABI, компилятора, компоновщика и рантайма одновременно.

Как неиспользуемая функция исчезает из программы

ДОСТИЖИМОСТЬ

```
компилятор создал символ F
↓ объектные данные содержат F
↓ компоновщик проходит от корней
↓ до F нет допустимого пути
↓ F не попадает в живой бинарник
```

ИНЖЕНЕРНЫЙ ВЫВОД

Скомпилированная функция еще не обязательно присутствует в executable.

Размер бинарника и RSS нельзя приравнивать

УПРОЩЕНИЕ

Удалили 10 MiB кода — файл на диске уменьшился.

ТОЧНАЯ МОДЕЛЬ

Физическая резидентность зависит от загрузки страниц, page кеш и того, к какому коду реально обращались.

ИНЖЕНЕРНЫЙ ВЫВОД

Размер файла, отображенная память и RSS — разные метрики.

Компоновка влияет и на запросы, и на сборку

Разделяем стоимость исполнения, объем бинарника и задержку CI.

```
CPU / регистры / кеш инструкций
код / типовые метаданные
время компиляции и линковки
```

Где ABI может влиять на горячий путь

ЗАДАЧА

Много маленьких вызовов, широкие сигнатуры и переходы через обертки создают перемещения регистров и данных.

МЕХАНИЗМ

При небольшом числе вызовов эффект часто несущественен. Под большой частотой его уже можно измерить.

ИНЖЕНЕРНЫЙ ВЫВОД

Без профиля оптимизировать ABI-границы не нужно.

Где обобщения могут влиять на горячий путь

ЗАДАЧА

После компиляции могут остаться словарные загрузки, методные обертки или косвенные вызовы.

МЕХАНИЗМ

Дополнительная специализация может увеличить рабочий набор инструкций.

ИНЖЕНЕРНЫЙ ВЫВОД

Измеряем конкретную инстанциацию, а не сам факт использования обобщений.

Где компоновщик влияет на эксплуатацию

ЗАДАЧА

На этапе поставки важны размер исполняемого файла, время компоновки, число символов и объем отладочных и типовых данных.

МЕХАНИЗМ

Для приложения важны трассировки стека, рефлексия и корректность связанных методов.

ИНЖЕНЕРНЫЙ ВЫВОД

Стоимость процесса сборки и обработки запросов измеряют отдельно.

Инцидент: вырос размер исполняемого файла

РАЗМЕР

```
сравнить `ls -lh` / размер артефакта
↓ go tool nm -size
↓ крупные тела форм / обертки?
↓ рефлексия / типовые метаданные / встроенные данные?
↓ выделить реальный источник роста
```

ИНЖЕНЕРНЫЙ ВЫВОД

> Одна фраза «Go-компилятор раздул бинарник» не является диагнозом.

Инцидент: обобщенный вызов стал медленнее

CPU

```
бенчмарк и CPU профиль
↓ конкретное общее тело / обертка
↓ `-m=2` / `objdump`
↓ проверка инлайнинга и словаря
↓ изменение → повторное измерение
```

ИНЖЕНЕРНЫЙ ВЫВОД

Нужно доказать, какая часть скрытой механики действительно осталась в горячем пути.

Инцидент: вызов делает много обращений к стеку

СТЕК

```
горячая функция
↓ большой аргумент или [N]T?
↓ исчерпаны ABI-регистры?
↓ взят адрес / spill / обертка?
↓ проверить ассемблер и профиль
```

ИНЖЕНЕРНЫЙ ВЫВОД

Стековая передача аргумента не означает автоматическую аллокацию в хипе.

Медленная компоновка — отдельное узкое место

ЗАДАЧА

Большой граф символов, релокации, типовые метаданные и множество инстанциаций увеличивают объем работы компоновщика.

МЕХАНИЗМ

Симптом проявляется во времени сборки CI и потреблении CPU/RAM, не обязательно в HTTP p99.

ИНЖЕНЕРНЫЙ ВЫВОД

Время подготовки артефакта — самостоятельная эксплуатационная метрика.

Почему рефлексию нельзя удалять по подозрению

УПРОЩЕНИЕ

В бинарнике много методных символов, и проект активно использует рефлексию.

ТОЧНАЯ МОДЕЛЬ

Только сравнение достижимости и размера покажет, что именно удерживается динамическим вызовом.

ИНЖЕНЕРНЫЙ ВЫВОД

Сначала измерить влияние, потом менять архитектуру.

Сборка символы ассемблер наблюдение

Эксперименты должны доказать реальные ABI-пути, словари и решения компоновщика.

```
-S / nm / objdump
Go ABI / обобщения / компоновщик
обнаружить → изменить → проверить
```

Эксперимент 1: регистровый ABI

GO / КОМАНДА

```
//go:noinline
func add(a, b int) int {
    return a + b
}
```

ЧТО ПРОИСХОДИТ

Собрать с `-gcflags=-S`, найти физические входные регистры и расположение результата на amd64.

ИНЖЕНЕРНЫЙ ВЫВОД

Доказать регистр-based вызов обычной функции Go.

Эксперимент 2: `string` и slice

GO / КОМАНДА

```
//go:noinline
func f(s string, b []byte) int {
    return len(s) + len(b)
}
```

ЧТО ПРОИСХОДИТ

В ассемблере сопоставить указатель и длину строки, указатель/длину/емкость среза с ABI-компонентами.

ИНЖЕНЕРНЫЙ ВЫВОД

Один логический параметр может занимать несколько регистров.

Эксперимент 3: `[1]T` против `[2]T`

GO / КОМАНДА

```
func one(a [1]uintptr) uintptr {
return a[0] }
func two(a [2]uintptr) uintptr {
return a[0] }
```

ЧТО ПРОИСХОДИТ

Добавить `//go:noinline`, проверить назначение аргументов. Для второго массива текущий ABI использует стек.

ИНЖЕНЕРНЫЙ ВЫВОД

Размер массива меняет calling convention.

Эксперимент 4: широкая структура

GO / КОМАНДА

```
type Big struct {
    A,B,C,D,E,F,G,H,I,J int
}
```

ЧТО ПРОИСХОДИТ

Передать в неинлайнимую функцию и сравнить расположение аргумента на amd64 и arm64.

ИНЖЕНЕРНЫЙ ВЫВОД

Бюджет доступных регистров зависит от архитектуры.

Эксперимент 5: адрес аргумента

GO / КОМАНДА

```
//go:noinline
func f(x int) *int {
    return &x
}
```

ЧТО ПРОИСХОДИТ

Посмотреть эскейп-анализ и физическое размещение значения. Отличать стековый слот ABI от аллокации в хипе.

ИНЖЕНЕРНЫЙ ВЫВОД

ABI spill и эскейп-анализ отвечают на разные вопросы.

Эксперимент 6: вызов замыкания

GO / КОМАНДА

```
func makeAdder(x int) func(int)
int {
return func(y int) int { return
x+y }
}
```

ЧТО ПРОИСХОДИТ

Через `objdump` найти вызов с контекстом захваченного `x`, на amd64 сопоставить со служебным регистром.

ИНЖЕНЕРНЫЙ ВЫВОД

Замыканию нужен скрытый контекст помимо обычных аргументов.

Эксперимент 7: увидеть символы форм

GO / КОМАНДА

```
//go:noinline
func Id[T any](x T) T { return x }
```

ЧТО ПРОИСХОДИТ

Вызвать функцию с несколькими типами и выполнить `go tool nm ./app | grep shape`. Число машинных тел заранее не предполагаем.

ИНЖЕНЕРНЫЙ ВЫВОД

Инстанциаций может быть больше, чем машинных тел.

Эксперимент 8: `dictionary` при `any(x)`

GO / КОМАНДА

```
//go:noinline
func Box[T any](x T) any { return
x }
```

ЧТО ПРОИСХОДИТ

Вызвать `Box[int]` и `Box[string]`, найти тела по форме, словари и ссылки на описатели типов.

ИНЖЕНЕРНЫЙ ВЫВОД

Конкретное слово Type для интерфейса приходит из типовых метаданных.

Эксперимент 9: вложенный обобщенный вызов

GO / КОМАНДА

```
func Inner[T any](x T) T { return
x }
func Outer[T any](x T) T {
    return Inner[T](x)
}
```

ЧТО ПРОИСХОДИТ

Отключить инлайнинг, найти ссылки на словарь вложенной инстанциации в символах и сгенерированных вызовах.

ИНЖЕНЕРНЫЙ ВЫВОД

Вызов обобщения может потребовать перехода между словарями.

Эксперимент 10: метод через `fmt.Stringer`

GO / КОМАНДА

```
type Box[T any] struct { V T }
func (b Box[T]) String() string {
    return fmt.Sprint(b.V)
}
```

ЧТО ПРОИСХОДИТ

Передать `Box[int]` как `fmt.Stringer`. Найти путь через `ITab`, обертку и соответствующее общее тело.

ИНЖЕНЕРНЫЙ ВЫВОД

Методная таблица должна получить вызываемый код конкретной инстанциации.

Эксперимент 11: методы с параметрами типов

ЗАДАЧА

По исходной лекции для Go 1.27 рассмотреть метод конкретного типа с собственным параметром типа.

МЕХАНИЗМ

Найти сгенерированные символы через `nm/objdump` и сравнить с обычной обобщенной функцией.

ИНЖЕНЕРНЫЙ ВЫВОД

Выводы о новых конструкциях привязываем к версии Go.

Эксперимент 12: выкинуть недостижимую функцию

GO / КОМАНДА

```
//go:noinline
func Used() {}
//go:noinline
func NeverCalled() {}
```

ЧТО ПРОИСХОДИТ

Вызвать только `Used`, собрать бинарник и проверить через `go tool nm`, присутствует ли `NeverCalled`.

ИНЖЕНЕРНЫЙ ВЫВОД

Удаление кода на уровне компоновщика можно наблюдать руками.

Эксперимент 13: интерфейс удерживает метод

GO / КОМАНДА

```
type Runner interface { Run() }
type T struct{}
func (T) Run() {}
```

ЧТО ПРОИСХОДИТ

Использовать T через интерфейс, затем посмотреть список сохраненных методных символов.

ИНЖЕНЕРНЫЙ ВЫВОД

Достижимость через интерфейс не равна прямому CALL из исходника.

Эксперимент 14: рефлексия и достижимость

GO / КОМАНДА

```
reflect.TypeOf(x).MethodByName("Run")
```

ЧТО ПРОИСХОДИТ

Сравнить бинарники с динамическим выбором метода и без него: `go tool nm -size` и общий размер.

ИНЖЕНЕРНЫЙ ВЫВОД

Рефлексия может расширить множество методов, которые нужно сохранить.

Эксперимент 15: бинарник без отладочных символов

GO / КОМАНДА

```
go build -o app-normal .
go build -ldflags='-s -w' -o
app-stripped .
```

ЧТО ПРОИСХОДИТ

Сравнить размер, результат `nm`, трассировку panic и возможности отладчика.

ИНЖЕНЕРНЫЙ ВЫВОД

DWARF и собственные таблицы Go runtime — разные системы.

Эксперимент 16: pclntab и трассировка стека

GO / КОМАНДА

```
func deep() { panic("boom") }
// runtime.Callers / CallersFrames
```

ЧТО ПРОИСХОДИТ

Получить стек panic, затем вручную сопоставить адреса кадрам функций и строкам.

ИНЖЕНЕРНЫЙ ВЫВОД

PC становится читаемым благодаря метаданным инструментария.

Эксперимент 17: размер обобщенного кода

ЗАДАЧА

Сравнить бинарник с одной инстанциацией обобщенной функции и с десятками разных типов.

МЕХАНИЗМ

Измерить размер символов общих тел, словарей и всего бинарника через `nm -size`.

ИНЖЕНЕРНЫЙ ВЫВОД

Рост размера не обязан быть линейным по числу типов.

Эксперимент 18: две архитектуры

GO / КОМАНДА

```
GOOS=linux GOARCH=amd64 go build
GOOS=linux GOARCH=arm64 go build
```

ЧТО ПРОИСХОДИТ

Сравнить физические ABI-регистры, набор символов и сгенерированный код одной программы.

ИНЖЕНЕРНЫЙ ВЫВОД

Одна семантика Go допускает разный машинный путь.

Язык обещает поведение, инструментарий выбирает реализацию

Отделяем спецификацию Go от конкретной компиляции и компоновки Go 1.27.x.

семантика вызова / обобщения / интерфейс
ABIInternal / форма / словарь / символы
измерение на фактическом бинарнике

Что гарантирует Go и что не гарантирует

УПРОЩЕНИЕ

Язык определяет вызовы, результаты, замыкания, обобщения и интерфейсы.

ТОЧНАЯ МОДЕЛЬ

Не обещает конкретные регистры, `ABIInternal`, словари, внутренние имена общих тел и точный состав символов.

ИНЖЕНЕРНЫЙ ВЫВОД

Корректность строим на языке/API; внутренности используем для расследования.

ABIInternal и ABI0 — разная стабильность

УПРОЩЕНИЕ

`ABI0` — документированная граница Go-ассемблера.

ТОЧНАЯ МОДЕЛЬ

`ABIInternal` может меняться между версиями компилятора; пользовательский asm не должен зависеть от его текущих регистров.

ИНЖЕНЕРНЫЙ ВЫВОД

Не смешиваем публичный двоичный контракт и внутренний.

Обобщения: семантика и генерация кода

УПРОЩЕНИЕ

Go гарантирует результат обобщенной функции для допустимых аргументов типа.

ТОЧНАЯ МОДЕЛЬ

Стратегия форм и словарей, число машинных тел и обертки — детали текущей реализации.

ИНЖЕНЕРНЫЙ ВЫВОД

Не используем внутренние имена символов как API.

Компоновщик: наблюдаемость и гарантии

УПРОЩЕНИЕ

Приложение должно корректно вызывать нужные функции и использовать отражение.

ТОЧНАЯ МОДЕЛЬ

Алгоритм достижимости, релокации и размещение символов — детали инструментария.

ИНЖЕНЕРНЫЙ ВЫВОД

Можно исследовать `.text`, `pclntab` и DWARF, не завися от их конкретной раскладки.

Финальная модель 4.4

ГОТОВЫЙ АРТЕФАКТ

```
Go-функция → ABIInternal
обобщение → форма + словарь
↓ скрытые аргументы / обертки
символы + релокации
↓ анализ достижимости / размещение
компоновщик → исполняемый файл
```

ИНЖЕНЕРНЫЙ ВЫВОД

Теперь понятен переход от отдельного машинного тела к работающему набору функций.

Главный инженерный вывод

УПРОЩЕНИЕ

Одна строка `F[int](x)` выглядит как обычный вызов с указанным типом.

ТОЧНАЯ МОДЕЛЬ

Физически это тело по форме, метаданные инстанциации, ABI, символы и позднее связывание адресов.

ИНЖЕНЕРНЫЙ ВЫВОД

Исследуем сгенерированный код и граф символов компоновщика, когда профиль или размер показывают проблему.

Следующая граница: загрузчик ОС

ЧТО ЗАКОНЧИЛИ

Компоновщик создал ELF/Mach-O/PE с кодом, данными и таблицами рантайма.

ЧТО ДАЛЬШЕ

Дальше ОС построит виртуальные отображения, разрешит динамические зависимости и передаст управление точке входа.

ИНЖЕНЕРНЫЙ ВЫВОД

Это следующая 4.5, не продолжение устройства ABI или словарей.